

JAVA WEB

Unit 6: Spring JPA

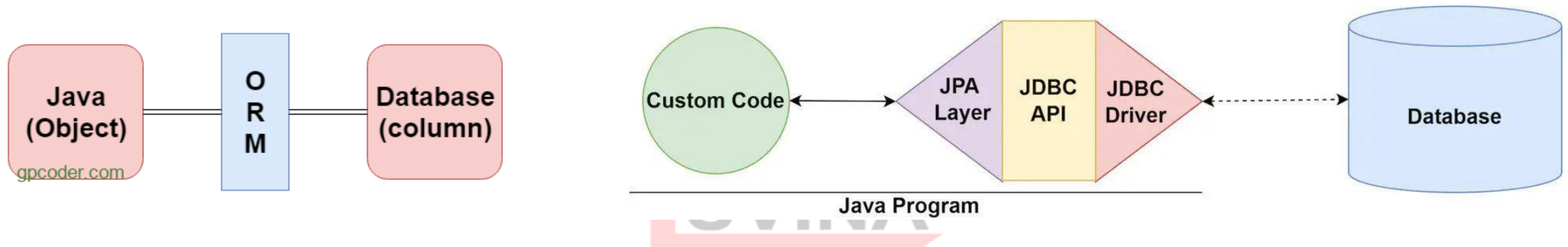


Mục lục

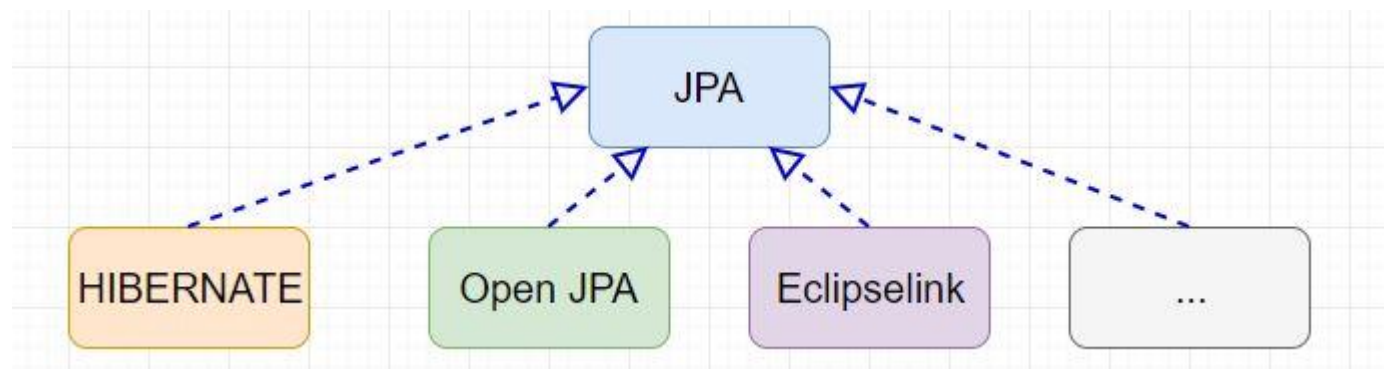
1. JPA, Object-Relation Mapping (ORM)
2. Spring JPA
 - Tạo Model
 - Tạo Repo (CrudRepository, JpaRepository)
 - Tạo Service sử dụng Optional
 - Chạy ứng dụng
3. Thư viện Lombok
4. Các dạng mapping: OneToOne, OneToMany, ManyToMany
5. Native SQL, JPQL
6. Giới thiệu về các dạng dữ liệu: DTO, Entity, Model.
7. **MapStruct**, ví dụ mapping giữa Entity và DTO

JPA, ORM

JPA là viết tắt của **Java Persistence API**, nó là một đặc tả Java cho việc ánh xạ giữa các đối tượng Java với cơ sở dữ liệu quan hệ sử dụng công nghệ phổ biến là **ORM** (Object Relational Mapping).



Triển khai JPA: Một số framework triển khai JPA phổ biến như: **Hibernate**, **EclipseLink**, **DataNucleus**, v.v. Nó còn được gọi là công cụ **Object-Relation Mapping (ORM)**.



Kiến trúc JPA

Entity: là các đối tượng thể hiện tương ứng với 1 table trong cơ sở dữ liệu.

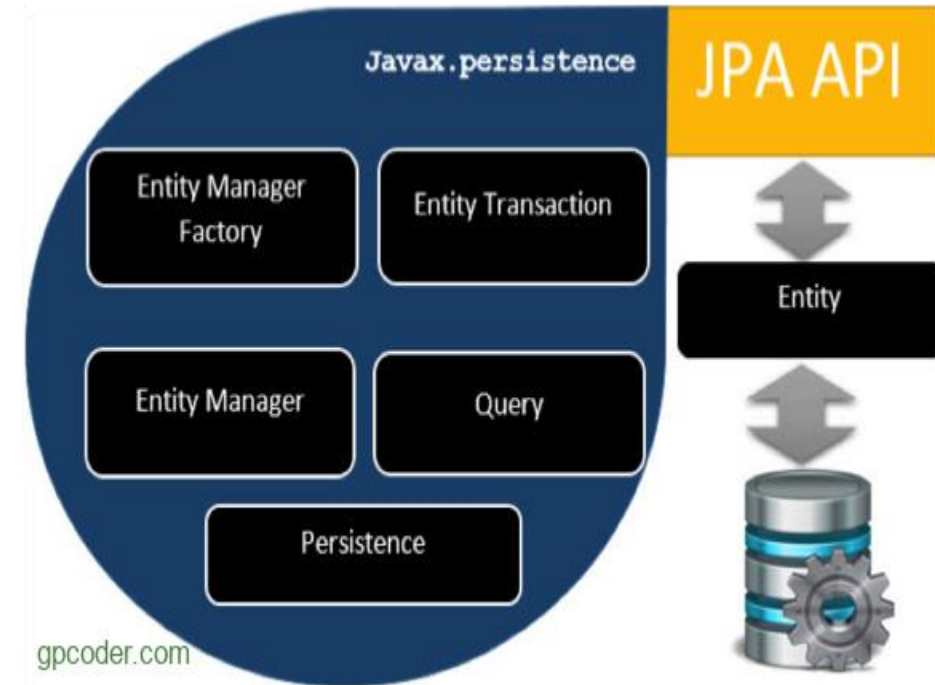
EntityManager: là một interface cung cấp các API cho việc tương tác với các Entity, như: Persist: lưu một thực thể mới tạo vào cơ sở dữ liệu; Merge: cập nhật trạng thái của entity vào cơ sở dữ liệu; Remove: xóa một instance của entity.

EntityManagerFactory: tạo ra một instance của EntityManager.

Persistence: định nghĩa một tập hợp các Entity class được quản lý bởi 1 instance của EntityManager trong ứng dụng.

EntityTransaction: Một Transaction là một tập hợp các thao tác trong đó tất cả các thao tác phải được thực hiện thành công hoặc tất cả thất bại. Đối tượng EntityManager giúp tạo EntityTransaction.

Query: This interface is implemented by each JPA vendor to obtain relational objects that meet the criteria.



Spring Boot JPA

Spring Boot JPA là một phần trong hệ sinh thái Spring Data, nó tạo ra một layer ở giữa tầng service và database, giúp chúng ta thao tác với database một cách dễ dàng hơn, tự động config và giảm thiểu code thừa thãi.

Spring Boot JPA đã wrapper Hibernate và tạo ra một interface mạnh mẽ.

Cài đặt: Để thêm Spring JPA vào project, bạn cần thêm dependency **spring-boot-starter-data-jpa**.

Ngoài ra, để connect tới MySql, SQL Server, Oracle chúng ta cần driver tương ứng vào pom.xml.

Tạo project

1. Tạo project sử dụng <https://start.spring.io/>
2. Thêm các dependencies: Spring Data JPA và (MySQL) MS SQL Server
3. Nhấp vào nút Generate
4. Mở bằng IntelliJ (Netbeans,....)



Tạo project (MySQL)

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <scope>runtime</scope>
  </dependency>
  <!-- JSP -->
  <dependency>
    <groupId>org.apache.tomcat.embed</groupId>
    <artifactId>tomcat-embed-jasper</artifactId>
  </dependency>
  <!-- jstl for jsp -->
  <dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>jstl</artifactId>
  </dependency>
</dependencies>
```

```
spring.mvc.view.prefix: /
spring.mvc.view.suffix: .jsp
server.port=8081
spring.datasource.url=jdbc:mysql://localhost:3306/SpringJPA?useSSL=false
spring.datasource.username=root
spring.datasource.password=root
```

```
## Hibernate Properties
# The SQL dialect makes Hibernate generate better SQL for the chosen database
spring.jpa.properties.hibernate.dialect = org.hibernate.dialect.MySQL5InnoDBDialect
# Hibernate ddl auto (create, create-drop, validate, update)
spring.jpa.hibernate.ddl-auto = update

logging.level.org.hibernate = ERROR
```

Tạo project (SQL Server)

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>

  <dependency>
    <groupId>com.microsoft.sqlserver</groupId>
    <artifactId>mssql-jdbc</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

```
spring.datasource.url= jdbc:sqlserver://localhost;databaseName=SpringJPA
spring.datasource.username= sa
spring.datasource.password= sa
```

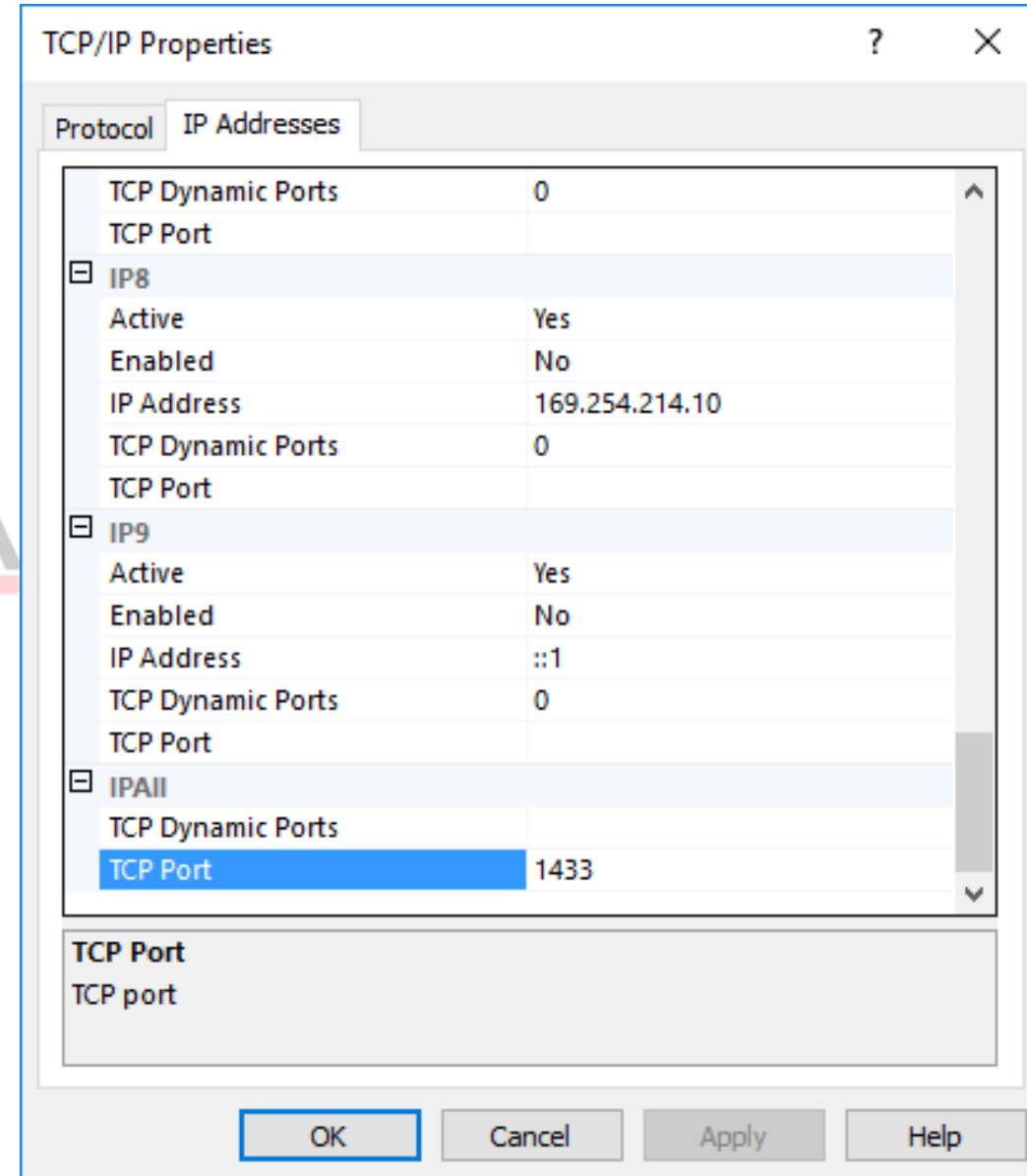
```
spring.jpa.properties.hibernate.dialect= org.hibernate.dialect.SQLServer2012Dialect
spring.jpa.hibernate.ddl-auto= update
```


Click **Protocol for SQLEXPRESS** under **SQL Server Network Configuration** on the left pane. On the right pane, right-click **TCP/IP**, and select **Properties**.

Scroll down to locate the **IPALL** node. Remove any value, if present for **TCP Dynamic Ports** and specify **1433** for **TCP Port**.

Again right-click **TCP/IP** on the right pane, and select **Enable**.

On the **SQL Server Services** node, right-click **SQL Server**, and select **Restart**.



JpaRepository

Để sử dụng Spring JPA, cần sử dụng interface JpaRepository.

Yêu cầu của interface này đó là phải cung cấp 2 thông tin:

- Entity (Đối tượng tương ứng với Table trong DB)
- Kiểu dữ liệu của khóa chính (primary key)

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface UserRepo extends JpaRepository<UserRecord, Integer> {
}
```

```
@Entity
@Table(name = "users")
@Data
public class UserRecord implements Serializable {
    private static final long serialVersionUID = -297553281792804396L;

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    // Mapping thông tin biến với tên cột trong Database
    @Column(name = "hp")
    private int hp;
    @Column(name = "stamina")
    private int stamina;

    // Nếu không đánh dấu @Column thì sẽ mapping tự động theo tên biến
    private int atk;
    private int def;
    private int agi;
}
```

```
import com.demoJPA.model.UserRecord;
import org.springframework.data.repository.CrudRepository;

public interface UserRepo extends CrudRepository<UserRecord, Integer> {}
```

```
@Service
public class UserService
{
    @Autowired
    private UserRepo userRepository;

    public List<UserRecord> getAllUsers() {
        List<UserRecord> userRecords = new ArrayList<>();
        userRepository.findAll().forEach(userRecords::add);
        return userRecords;
    }

    public void addUser(UserRecord userRecord) {
        userRepository.save(userRecord);
    }
}
```

```
@SpringBootApplication
public class La2022SpringBootJpaApplication {
    public static void main(String[] args) {
        ApplicationContext context = SpringApplication.run(La2022SpringBootJpaApplication.class, args);
        UserRepo userRepository = context.getBean(UserRepo.class);

        // Lấy ra toàn bộ user trong db
        userRepository.findAll().forEach(System.out::println);

        // Lưu user xuống database
        UserRecord user = userRepository.save(new UserRecord()); // Id tự sinh
        // Sau khi lưu xong, nó trả về User đã lưu kèm theo Id.
        System.out.println("User vừa lưu có ID: " + user.getId());
        Integer userId = user.getId();
        // Cập nhật user.
        user.setAgi(103);
        // Update user: Lưu ý, lúc này đối tượng user đã có Id.
        // Nên nó sẽ update vào đối tượng có Id này chứ không insert một bản ghi mới
        userRepository.save(user);

        // Query lấy ra user vừa xong để kiểm tra xem.
        UserRecord user2 = userRepository.findById(userId).get();
        System.out.println("User: " + user);
        System.out.println("User2: " + user2);
        // Xóa User khỏi DB:
        userRepository.delete(user);
        UserRecord user3 = userRepository.findById(userId).orElse(null);
        System.out.println("User3: " + user3);
    }
}
```

isPresent(); isEmpty(); ifPresent();

Optional được giới thiệu ở trong java 8, nó cung cấp những tùy chọn giá trị thay vì phải tham chiếu đến giá trị null!

```
public static void main(String[] args) {
    Optional<String> opt = Optional.of("Java Optional");
    if (opt.isPresent()) {
        System.out.println("Yes"); //
    } else {
        System.out.println("NO");
    }
    opt = Optional.ofNullable(null); // null => ko tồn tại
    if (opt.isPresent()) {
        System.out.println("Yes2");
    } else {
        System.out.println("NO2"); //
    }
    if (opt.isEmpty()) {
        System.out.println("Yes3"); //
    } else {
        System.out.println("NO3");
    }
    opt = Optional.of("Java Optional");
    // Thay vì phải kiểm tra !=null trước khi in
    opt.ifPresent(name -> System.out.println(name));
}
```

orElse(); orElseGet();

```
public static void main(String[] args) {  
    String nullName = null;  
    // Kiểm tra xem nullName có null hay ko, nếu nullName null thì gán Java cho name  
    String name = Optional.ofNullable(nullName).orElse("Java");  
    if (name.equals("Java")) {  
        System.out.println("Yes");  
    } else System.err.println("No");  
    /*  
    Method orElseGet() giống như orElse()  
    , nhưng thay vì truyền vào 1 giá trị, thì phải truyền vào một functional interface  
    */  
    name = Optional.ofNullable(nullName).orElseGet(() -> "Java");  
    if (name.equals("Java")) {  
        System.out.println("Yes2");  
    } else System.err.println("No2");  
}
```

Tạo lớp Service

Sử dụng các phương thức của Optional để kiểm tra dữ liệu

```
@Service
public class UserService {
    @Autowired
    private UserRepo userRepository;

    public UserRecord findById1(Integer id) throws Exception {
        return userRepository.findById(id).orElseThrow(() -> new Exception("Khong ton tai"));
    }

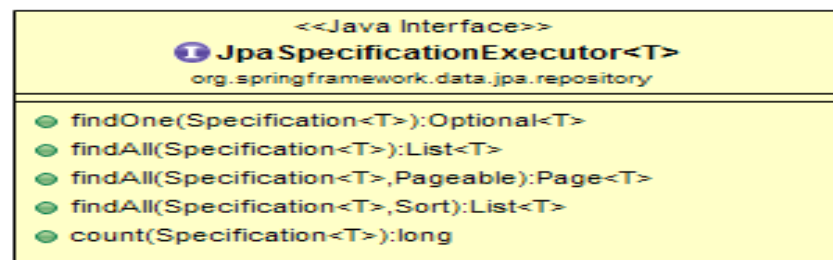
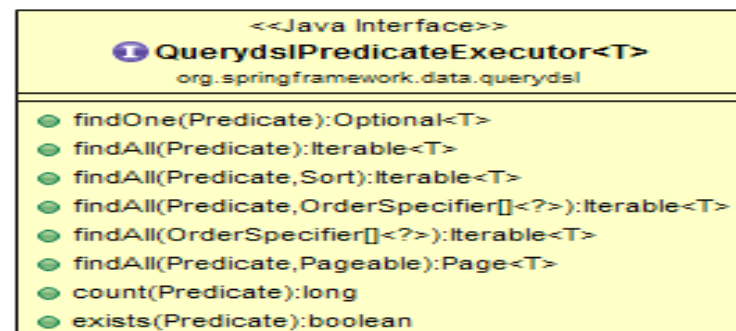
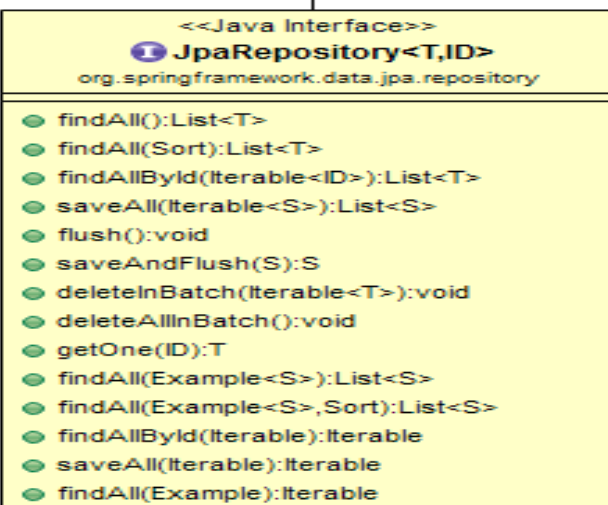
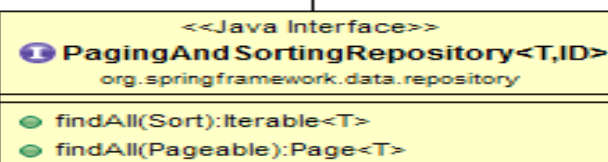
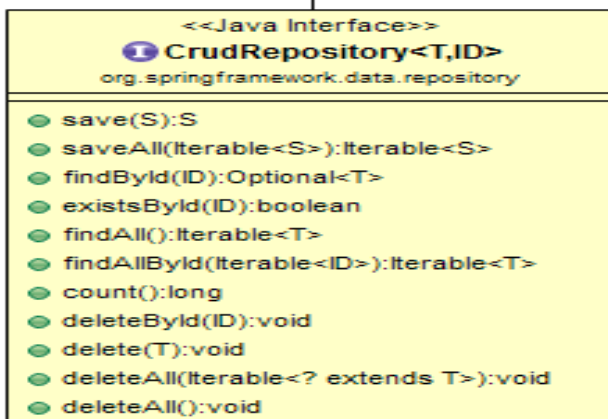
    public Optional<UserRecord> findById2(Integer id) {
        return userRepository.findById(id);
    }
}

@SpringBootApplication
public class La2022SpringBootJpaApplication1 {
    public static void main(String[] args) {
        ApplicationContext context = SpringApplication.run(La2022SpringBootJpaApplication1.class, args);
        UserService userService = context.getBean(UserService.class);
        try {
            UserRecord ur = userService.findById1(1000);
        } catch (Exception ex) {
            System.out.println(ex.getMessage());
        }
        Optional<UserRecord> ur2 = userService.findById2(1000);
        if (ur2.isPresent()) {
            System.out.println(ur2.toString());
        } else System.out.println("Ko ton tai");
    }
}
```

CrudRepository<T, ID extends Serializable> interface

```
public interface CrudRepository < T, ID > extends Repository < T, ID > {  
    <S extends T > S save(S entity);  
    <S extends T > Iterable < S > saveAll(Iterable < S > entities);  
    Optional < T > findById(ID id);  
    boolean existsById(ID id);  
    Iterable < T > findAll();  
    Iterable < T > findAllById(Iterable < ID > ids);  
    long count();  
    void deleteById(ID id);  
    void delete(T entity);  
    void deleteAll();  
}
```





interface JpaRepository

```
public interface JpaRepository < T, ID > extends PagingAndSortingRepository < T, ID > , QueryByExampleExecutor < T > {  
    List < T > findAll();  
    List < T > findAll(Sort sort);  
    List < T > findAllById(Iterable < ID > ids);  
    <S extends T> List < S > saveAll(Iterable < S > entities);  
    void flush();  
    <S extends T> S saveAndFlush(S entity);  
    void deleteInBatch(Iterable < T > entities);  
    void deleteAllInBatch();  
    T getOne(ID id);  
    @Override  
        <S extends T> List < S > findAll(Example < S > example);  
    @Override  
        <S extends T> List < S > findAll(Example < S > example, Sort sort);  
}
```

Lombok

Lombok là một công cụ generate code một cách tự động. Nhưng khác với các IDEs làm, Lombok generate các phương thức trong các tập tin **.class**.

@Data: bao gồm **@ToString** , **@EqualsAndHashCode** , **@Getter**, **@Setter**.

@ToString: sẽ generate phương thức **toString()**.

@EqualsAndHashCode: sẽ generate phương thức **hashCode()** và **equals()**.

@Getter/ **@Setter**: sẽ generate phương thức **getter** / **setter** cho các field.

@NoArgsConstructor: sẽ generate một hàm construct không có đối số.

@AllArgsConstructor: sẽ generate một hàm construct có đối số của tất cả field có trong class.

OneToOne

Hai lớp model

```

@Entity
@Table(name = "address")
public class Address {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    @Column(name = "address_id")
    private Long id;
    private String street;
    private String city;

    @OneToOne(mappedBy = "address")
    @JsonBackReference
    private User user;

    public Address() {
    }

```

```

@Entity
@Table(name = "user")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private Long id;

    private String name;

    private String phone;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "address_id", referencedColumnName = "address_id")
    @JsonManagedReference
    private Address address; //biến này phải trùng với mappedBy trong lớp Address

```

OneToOne

Các giao diện, lớp: Repo, Services

```
public interface UserRepository extends JpaRepository<User, Long> {  
}  
  
public interface UserService {  
    void save(User user);  
    User findById(Long id);  
}  
  
@Service  
public class UserServiceImpl implements UserService {  
  
    @Autowired  
    private UserRepository userRepository;  
  
    @Override  
    public void save(User user) {  
        this.userRepository.save(user);  
    }  
  
    @Override  
    public User findById(Long id) {  
        return this.userRepository.findById(id).orElse(null);  
    }  
}
```

OneToOne

```
@RestController
public class UserController {

    @Autowired
    private UserService userService;

    @GetMapping("/registUser")
    public void registUser() {
        Address address = new Address( street: "Cau Giay", city: "Ha Noi");

        User user = new User( name: "Nguyen Van An", phone: "09012345678");
        user.setAddress(address);

        this.userService.save(user);
    }

    @GetMapping("/getUser/{id}")
    public User getUser(@PathVariable Long id) { return this.userService.findById(id); }
}
```

Filter objects

- Functions
 - dbspringj
 - demojsf
 - demomybatis
 - employee_management_system
 - example_db
 - hibernateonetomany
 - mybatisannotation
 - onetomany
 - org_db
 - sakila
 - sampledb
 - springbatd
 - springjpa
- Tables
 - address
 - cart
 - course
 - employee
 - hibernate_sequence
 - items
 - product
 - student
 - student_course
 - user
 - users

1 • SELECT * FROM springjpa.user;

Result Grid

	id	name	phone	address_id
▶	1	Nguyen Van A	09012345678	4
	2	Nguyen Van A	09012345678	5
	3	Nguyen Van An	09012345678	6

user 1 ×

Demo: <http://localhost:8081/registUser>

OneToMany

Hai lớp model

```
@Entity
public class Cart {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "cart_id") //
    private Long cartId;

    @OneToMany(mappedBy = "cart", cascade = CascadeType.ALL)
    @JsonManagedReference
    private Set<Items> items;
```

```
@Entity
@Table(name = "items")
public class Items {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @ManyToOne
    @JoinColumn(name = "cart_id", nullable = false, referencedColumnName = "cart_id")
    @JsonBackReference
    private Cart cart; // biến này tên giống mappedBy trong lớp Cart
```

OneToMany

Các giao diện, lớp: Repo, Services

```
public interface CartRepository extends JpaRepository<Cart, Long> {  
}  
  
public interface CartService {  
    void save(Cart cart);  
    Cart findById(Long id);  
    public List<Cart> listAllCars();  
}  
  
@Service  
public class CartServiceImpl implements CartService {  
    @Autowired  
    private CartRepository cartRepository;  
    @Override  
    public void save(Cart cart) {  
        this.cartRepository.save(cart);  
    }  
    @Override  
    public Cart findById(Long id) {  
        return this.cartRepository.findById(id).orElse(null);  
    }  
    public List<Cart> listAllCars() {  
        return cartRepository.findAll();  
    }  
}
```


OneToMany

```
@RestController
public class CartController {
    @Autowired private CartService cartService;
    @GetMapping("/addCart")
    public void addCart() {
        Items item1 = new Items(name: "Item 1"); Items item2 = new Items(name: "Item 2");
        Set<Items> items = new HashSet<>(); items.add(item1); items.add(item2);

        Cart cart = new Cart();
        cart.setItems(items);

        item1.setCart(cart); item2.setCart(cart);
        // lưu dữ liệu xuống 2 bảng Cart và Items
        cartService.save(cart);
    }

    @GetMapping("/carts")
    public List<Cart> getALLCarts() {
        List<Cart> listCarts = cartService.listAllCars();
        return listCarts;
    }

    @GetMapping("/getCart/{id}")
    public Cart getCart(@PathVariable Long id) { return this.cartService.findById(id); }
}
```

Filter objects

- hibernateonetomany
- mybatisannotation
- onetomany
- org_db
- sakila
- sampledb
- springbatch
- springjpa
 - Tables
 - address
 - cart
 - course
 - employee
 - hibernate_sequence
 - items
 - product
 - student
 - student_course
 - user
 - users

1 • SELECT * FROM springjpa.items;

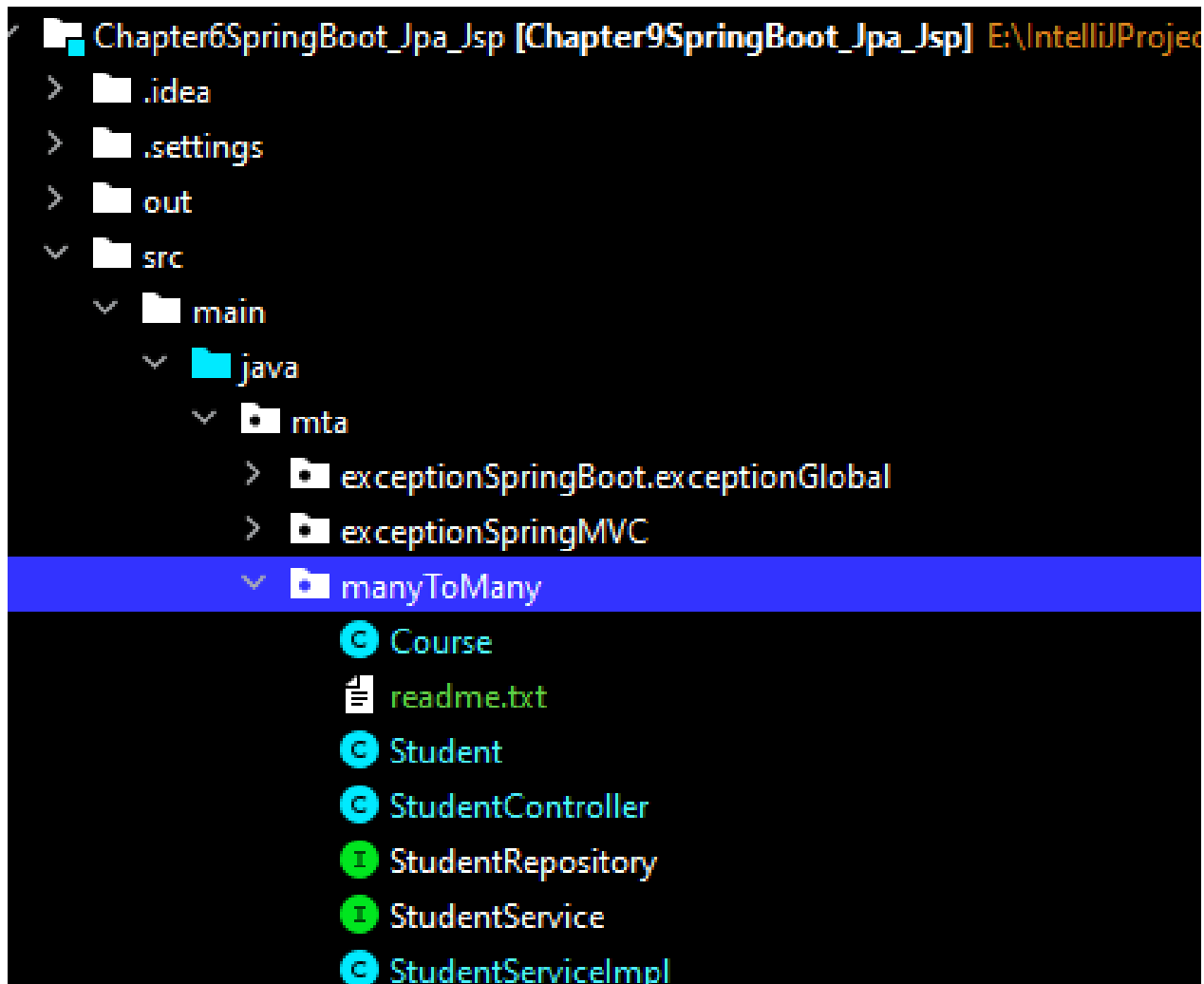
Result Grid

id	name	cart_id
1	Item 1	1
2	Item 2	1
3	Item 1	2
4	Item 2	2
NULL	NULL	NULL

Demo: <http://localhost:8081/addCart>

ManyToMany

Tham khảo tại

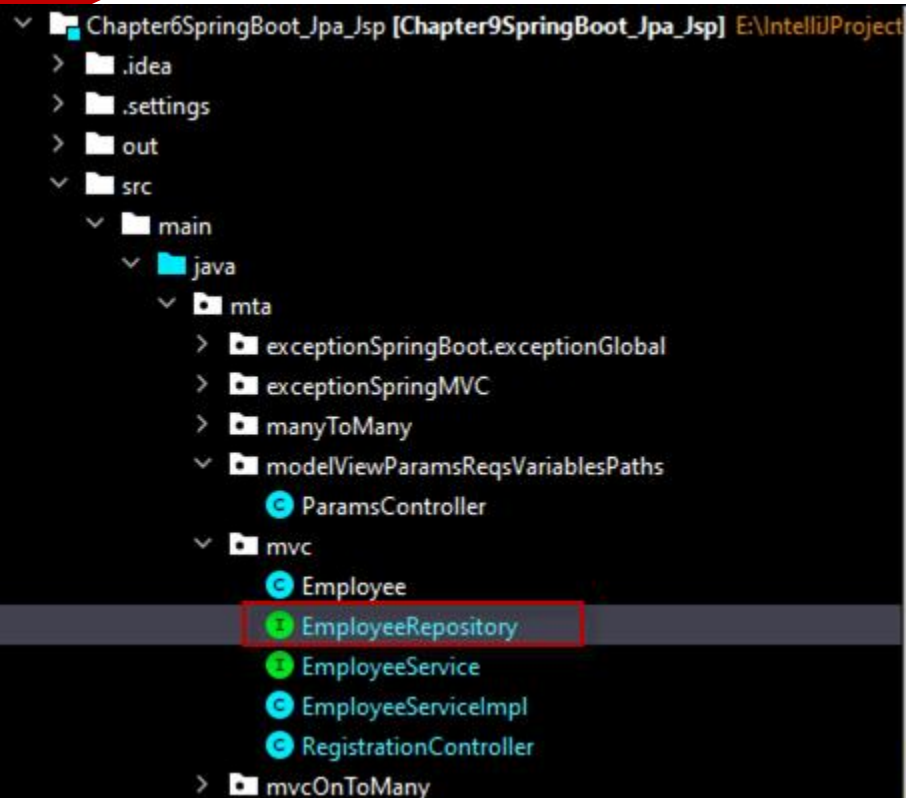


Native SQL, JPQL

Khi sử dụng Spring Data JPA thì mặc định sẽ sử dụng kiểu truy vấn là JPQL (Java Persistence Query Language) để thao tác dữ liệu. Cú pháp của JPQL tương tự như SQL (tham khảo thêm: JPQL_Chapter7.pdf)

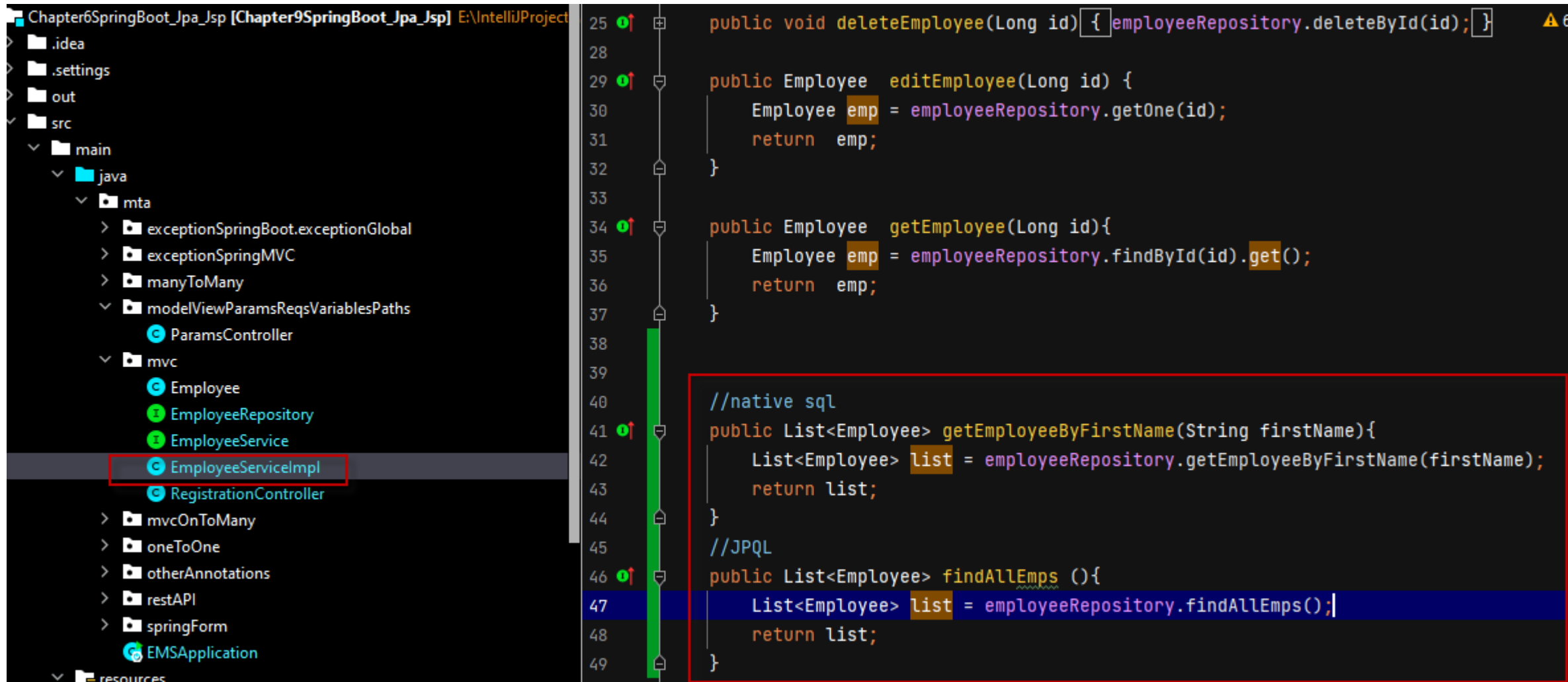
Tuy nhiên Spring Data JPA cũng hỗ trợ **native SQL**, điều này cho phép chúng ta thực hiện các câu lệnh truy vấn phức tạp để trích xuất thông tin cần thiết. Để định nghĩa một **native query**, chúng ta khai báo phương thức trong Repo với **annotation @Query** với câu truy vấn SQL và thuộc tính **nativeQuery=true**.

Native SQL, JPQL



```
1 package mta.mvc;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import org.springframework.data.jpa.repository.Query;
5 import org.springframework.stereotype.Repository;
6
7 import java.util.List;
8
9 @Repository
10 public interface EmployeeRepository extends JpaRepository<Employee, Long>{
11     // Native SQL
12     @Query(value="select * from Employee a where a.first_name= :firstName", nativeQuery=true)
13     List<Employee> getEmployeeByFirstName(String firstName);
14     // JPQL
15     @Query(value = "SELECT emp FROM Employee emp")
16     List<Employee> findAllEmps();
17 }
18
```

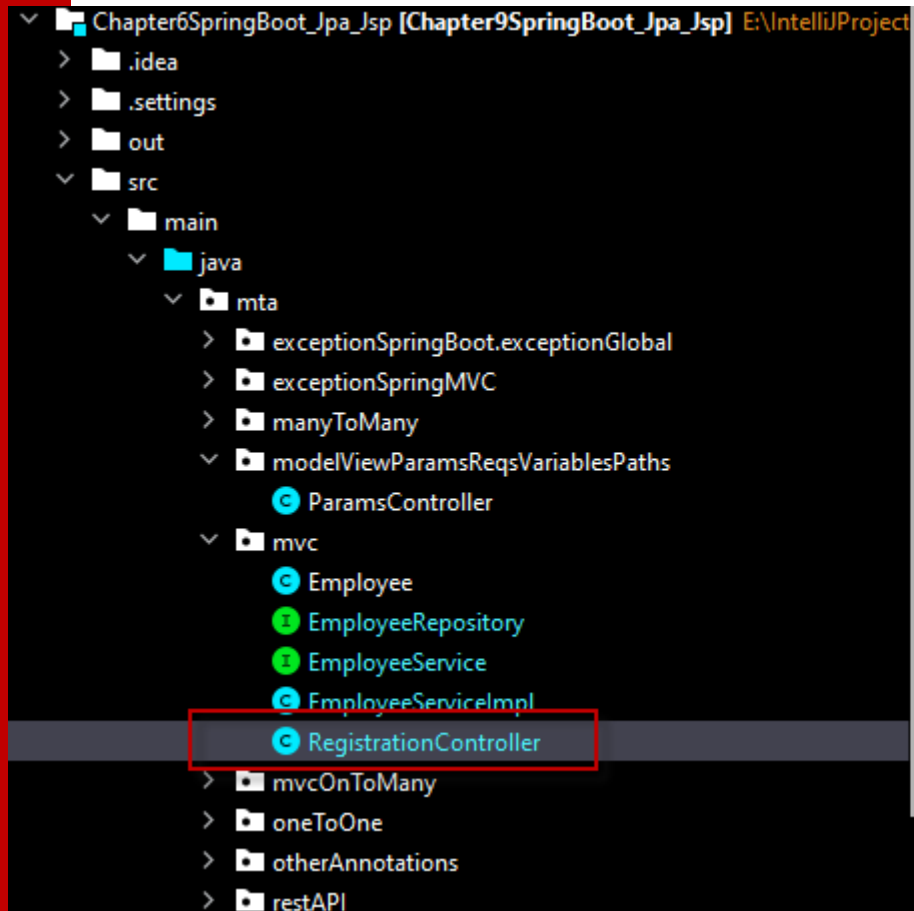
Native SQL, JPQL



```
Chapter6SpringBoot_Jpa_Jsp [Chapter9SpringBoot_Jpa_Jsp] E:\IntelliJProject
.idea
.settings
out
src
  main
    java
      mta
        exceptionSpringBoot.exceptionGlobal
        exceptionSpringMVC
        manyToMany
        modelViewParamsReqsVariablesPaths
          ParamsController
        mvc
          Employee
          EmployeeRepository
          EmployeeService
          EmployeeServiceImpl
          RegistrationController
        mvcOnToMany
        oneToOne
        otherAnnotations
        restAPI
        springForm
        EMSApplication
      resources

25 public void deleteEmployee(Long id){ employeeRepository.deleteById(id);}
28
29 public Employee editEmployee(Long id) {
30     Employee emp = employeeRepository.getOne(id);
31     return emp;
32 }
33
34 public Employee getEmployee(Long id){
35     Employee emp = employeeRepository.findById(id).get();
36     return emp;
37 }
38
39
40 //native sql
41 public List<Employee> getEmployeeByFirstName(String firstName){
42     List<Employee> list = employeeRepository.getEmployeeByFirstName(firstName);
43     return list;
44 }
45 //JPQL
46 public List<Employee> findAllEmps(){
47     List<Employee> list = employeeRepository.findAllEmps();
48     return list;
49 }
```

Native SQL, JPQL



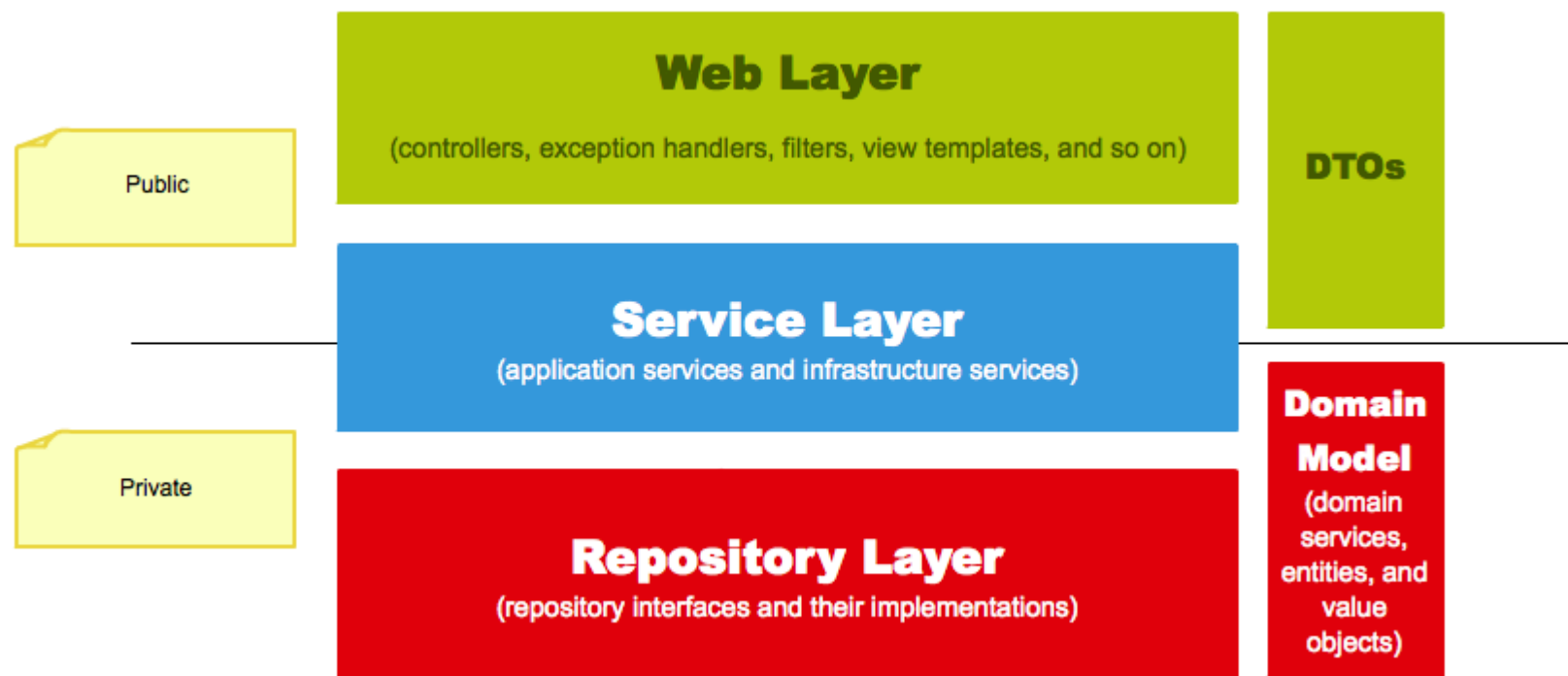
```
43
44
45 @GetMapping("/edit")
46 public String editEmployee(@RequestParam("id") String id, Map<String, Object> model) {
47     Employee emp= employeeService.editEmployee(Long.parseLong(id));
48     model.put("employee", emp );
49     return "views/Registration";
50 }
51 //===== Native SQL =====
52 @GetMapping("/listNativeSQL/{fName}")
53 public String listOfEmployeeNativeSQL(Model model, @PathVariable String fName) {
54     List<Employee> employeeList = employeeService.getEmployeeByFirstName(fName);
55     model.addAttribute("empList", employeeList);
56     return "views/employeeList";
57 }
58 //=====
59 @GetMapping("/listJPQL")
60 public String listOfEmployeeJPQL(Model model) {
61     List<Employee> employeeList = employeeService.findAllEmps();
62     model.addAttribute("empList", employeeList);
63     return "views/employeeList";
64 }
```

Demo:

<http://localhost:8081/listNativeSQL/fdfd>

<http://localhost:8081/listJPQL>

Các dạng dữ liệu



Trong mô hình 3 lớp của ứng dụng Spring boot thì mỗi layer sẽ có dạng data khác nhau, có nhiệm vụ, mục đích khác nhau. Ví dụ, trong sơ đồ trên thì **Controller** không nên đựng tới data dạng **domain model** hoặc **entity**, chỉ được phép nhận và trả về **DTO**.

Một trong những lý do phải chia thành nhiều dạng dữ liệu vì nếu để một dạng thì sẽ bị leak (lộ) các dữ liệu nhạy cảm. Ví dụ, dữ liệu về User sẽ có password là dữ liệu nhạy cảm.

Các dạng dữ liệu (3 dạng)

DTO (Data transfer object): là các class đóng gói data để chuyển giữa client - server hoặc giữa các service trong microservice. Mục đích tạo ra DTO là để giảm bớt lượng info không cần thiết phải chuyển đi, và cũng tăng cường độ bảo mật.

Domain model: là các lớp đại diện cho miền ứng dụng.

Entity: cũng là domain model nhưng tương ứng với **table** trong DB, có thể map vào DB được.

MapStruct

Khi phát triển một ứng dụng nhiều lớp trong java, việc mapping giữa các đối tượng Java Bean với nhau là điều không thể thiếu.

Ví dụ ánh xạ các entities trả về thành các DTO. Việc thực hiện mapping thủ công từ entities sang DTO là một công việc tẻ nhạt và dễ gây lỗi.

MapStruct được tạo ra để giải quyết vấn đề mapping dữ liệu giữa các Java Bean một cách tự động, thuận tiện và nhanh chóng. Để sử dụng MapStruct, chúng ta thêm dependency sau:

```
<dependency>  
    <groupId>org.mapstruct</groupId>  
    <artifactId>mapstruct</artifactId>  
    <version>1.5.1.Final</version>  
</dependency>
```

Ví dụ sử dụng MapsStruct thực hiện mapping

Xét lớp Entity và DTO khác nhau ở 02 trường startDate và endDate:

```
@Entity
@Table
public class Event {
    @Id
    @Column
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;
    @Column
    private String title;
    @Column
    private String url;
    private String clazz;
    @Column(name = "start_date")
    @Temporal(TemporalType.TIMESTAMP)
    private Date startDate;
    @Column(name = "end_date")
    @Temporal(TemporalType.TIMESTAMP)
    private Date endDate;
```

```
public class EventDto {
    private int id;
    private String title;
    private String url;
    private String clazz;
```

Các lớp: Mapper, Repo, Service

```
import org.mapstruct.Mapper;

import com.roytuts.springboot.mapstruct.jpa.entity.dto.mapping.dto.EventDto;
import com.roytuts.springboot.mapstruct.jpa.entity.dto.mapping.entity.Event;

@Mapper(componentModel = "spring")
public interface EventMapper {

    EventDto toEventDto(Event event);

    List<EventDto> toEventDtos(List<Event> events);

    Event toEvent(EventDto eventDto);

}
```

```
public interface EventRepository extends JpaRepository<Event, Integer>
{
}
```

```
@Service
```

```
public class EventService {
```

```
    @Autowired
```

```
    private EventRepository eventRepository;
```

```
    @Autowired
```

```
    private EventMapper eventMapper;
```

```
    public List<Event> getEvents() {
```

```
        return eventRepository.findAll();
```

```
    }
```

```
    public List<EventDto> getEventDtos() {
```

```
        return eventMapper.toEventDtos(eventRepository.findAll());
```

```
    }
```

```
    public EventDto getEventDto(Integer id) {
```

```
        return eventMapper.toEventDto(
            eventRepository.findById(id).orElseThrow());
```

```
    }
```

```
    public Event getEvent(Integer id) {
```

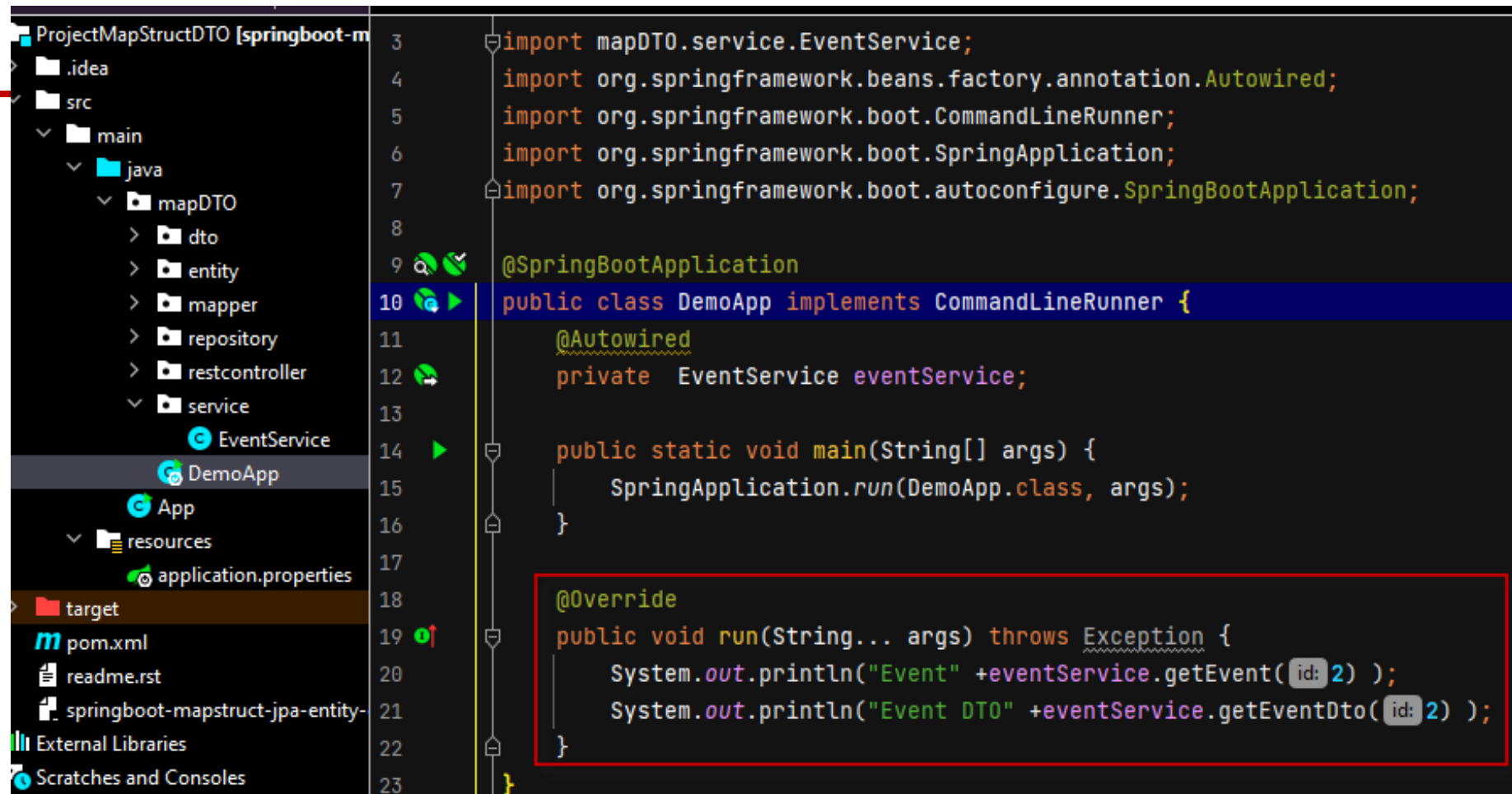
```
        return eventMapper.toEvent(getEventDto(id));
```

```
    }
```

```
}
```

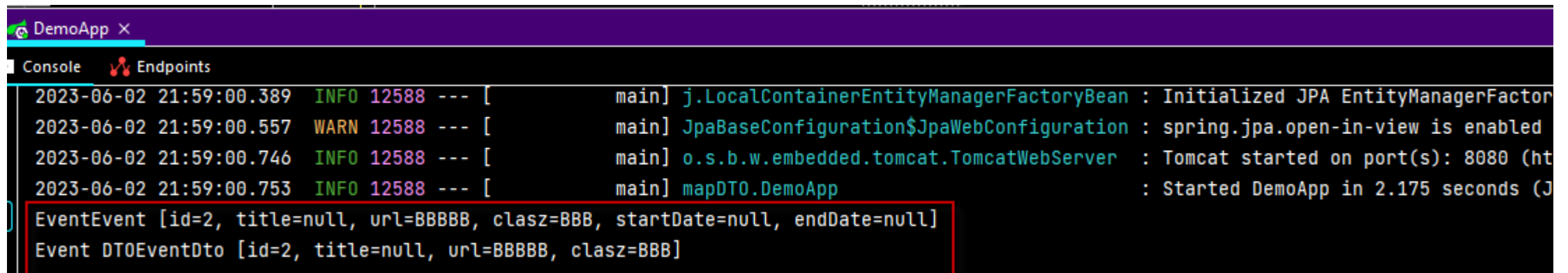
Demo

36



The screenshot shows an IDE with a project structure on the left and Java code on the right. The project structure includes a package hierarchy: `mapDTO` (containing `dto`, `entity`, `mapper`, `repository`, `restcontroller`, and `service`), and a `service` package containing `EventService`. The `DemoApp` class is highlighted in the project structure. The Java code on the right is as follows:

```
3  import mapDTO.service.EventService;
4  import org.springframework.beans.factory.annotation.Autowired;
5  import org.springframework.boot.CommandLineRunner;
6  import org.springframework.boot.SpringApplication;
7  import org.springframework.boot.autoconfigure.SpringBootApplication;
8
9  @SpringBootApplication
10 public class DemoApp implements CommandLineRunner {
11     @Autowired
12     private EventService eventService;
13
14     public static void main(String[] args) {
15         SpringApplication.run(DemoApp.class, args);
16     }
17
18     @Override
19     public void run(String... args) throws Exception {
20         System.out.println("Event" + eventService.getEvent(id: 2));
21         System.out.println("Event DTO" + eventService.getEventDto(id: 2));
22     }
23 }
```



The screenshot shows the IDE's console output for the `DemoApp` application. The console displays several log messages, including the application starting on port 8080. The output is as follows:

```
2023-06-02 21:59:00.389 INFO 12588 --- [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory
2023-06-02 21:59:00.557 WARN 12588 --- [main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled
2023-06-02 21:59:00.746 INFO 12588 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8080 (ht
2023-06-02 21:59:00.753 INFO 12588 --- [main] mapDTO.DemoApp : Started DemoApp in 2.175 seconds (J

EventEvent [id=2, title=null, url=BBBBB, clasz=BBB, startDate=null, endDate=null]
Event DTOEventDto [id=2, title=null, url=BBBBB, clasz=BBB]
```